When Does Bank Layout Pay? Function-Preserving Channel Relabeling and Tap Skew on a Lockstep Zero-Skipping FPGA CNN Core

Lee Yoo Joon
School of Computer Science and Engineering, Yeungnam University

Information and Communication Engineering Major

Gyeongsan, Republic of Korea

(e-mail)

(Advisor name)
(Dept. name of organization)

(Name of organization)

(City, Country)

(e-mail address)

Abstract—A zero-skipping convolution core whose input banks run in lockstep waits, in every window, for the bank with the most nonzero activations, so the assignment of window taps to banks sets its cycle count. On our integer CNN core for a Zynq XC7Z020, a condition on the layer shapes alone tells where no tap-to-bank layout can change the cycles. We compare three layouts that keep the bitstream and the integer output unchanged: a channel relabeling fitted offline and folded into the adjacent layers, a data-free skew that spreads each channel's taps over the banks, and the two combined. At four values per cycle, two relabelings of a 32-channel denoiser cut the core cycles measured on one image by 7.02% and 7.60%, as a model pushed before the export predicted to the cycle. In counts over 79 test images the skew saves 9.20% and the combination 9.93%, against 7.46% and 8.10% for the relabelings; under pre-registered protocols this order held on all nine networks whose channel count is a multiple of four, and no layout changed the count where the shape condition says it cannot.

Keywords—zero skipping, activation sparsity, load balancing, bank layout, channel permutation, FPGA, CNN accelerator, cycle model

I. INTRODUCTION

A core that skips zero activations saves multiplications, but lanes that advance together make each window cost as much as the busiest one, a stall Cnvlutin measures as its own category [1]. Known remedies for activation imbalance change the network or add hardware: S2TA bounds the nonzeros per block by discarding activations [2], and Sense rebalances channels at run time, noting that input maps "cannot be trained offline to balance IFM workload" [3]. Offline reorderings target weights: SparTen sorts filters by density and reorders the next layer's weights to match [4], and Pool and Yu fold into the preceding layer a channel permutation that lets N:M pruning keep more weight magnitude [5]. VARA permutes inputs offline from activation statistics to cluster co-zero inputs, reading them through an index unit [6], and skewed storage gives conflict-free parallel access to the rows and columns of a matrix [7]. Permutation, folding, profiling and skewing are thus not new; our search (Crossref, arXiv, web; October 2026) found no work that spreads co-active channels over lockstep zero-skipping banks by an exact, folded relabeling.

In our core, four tap banks run in lockstep, and the slot in which the host puts a tap decides its bank. We contribute (i) a shape-only condition necessary for any layout to change a layer's cycles, and its window-port regimes (Section II); (ii) three layouts that keep the bitstream and the integer output unchanged, with checks: a folded channel relabeling, a data-free skew and both combined (Section III); (iii) board cycles of two relabelings, predicted to the cycle (V-A); (iv) two pre-registered ten-network studies with bootstrap intervals (V-B, V-C); (v) fit time and a host-time bound (V-D).

II. CORE, COST MODEL AND SHAPE CONDITION

Core. The host builds each layer's windows (im2col), sends them with the layer's weights and requantizes between layers; the core computes every convolution with INT8 weights, 7-bit unsigned activations and INT32 accumulation. Its 64 multipliers form P = 16 output lanes times T = 4 tap banks; its window port takes W values per cycle (W = 2 only in RTL simulation). In sparse mode, used throughout, each bank processes only its nonzero taps; slot j of a window belongs to bank j mod 4.

Cost of a window. For a window with K taps and COUT outputs, and m the largest number of nonzero taps in one bank, the core spends C cycles:

$$C = \max(\lceil K/W \rceil + 1, \lceil COUT/P \rceil \cdot \max(1, m), COUT), \quad (1)$$

for loading the next window, computing $\lceil COUT/P \rceil$ output groups and emitting COUT results. Each layer adds a fill and drain of min(load, compute of the last window) + 3 + g cycles, g being the outputs in the last group of P, a rule read off board runs of four networks on one image; the first layer has a measured constant (17 cycles for the board network at W = 4). We call (1) summed over the windows the count, and the count plus the fill and drain the exact prediction.

When a layout can pay. A bank holds at most $\lceil K/T \rceil$ taps. No window is then compute-bound for any input, and no layout can change the count, if

$$\lceil COUT/P \rceil \cdot \lceil K/T \rceil \le \max(\lceil K/W \rceil + 1, COUT). \quad (2)$$

Failing (2) is thus necessary for a gain; without the rounding, (2) reads $COUT \le P \cdot T/W$, the ridge point of a roofline [8] whose bandwidth roof is the load. So 32-output hidden layers (K = 288: 144 against 145 at W = 2 and 73 at W = 4) can pay only at W = 4, layers with 45 or 64 outputs also at W = 2, and none at W = 1 or in the last layer ($COUT \le 4$). As a reference, perfect balance, $m = \lceil n/T \rceil$ for n nonzero taps, bounds every assignment of a window's taps to lockstep banks.

III. Three Function-Preserving Layouts

In (ky, kx, channel) order, tap $s \cdot N + p$ (s the 3×3 position, p the channel, N the input channels) is in bank p mod 4 when 4 divides N: in this conventional order, L0 or the Original, each bank holds N/4 whole channels (Fig. 1a).

Relabeling (L1). Let π map a position p of a hidden map to its original channel. The producer computes its output channels in the order π, with weight rows, biases and requantization scales reordered, and the consumer's weight column for tap $s \cdot N + p$ comes from channel π(p). As ReLU and requantization act per channel and the consumer adds the same INT32 products, the integer output is identical for every input, while π decides which channels share a bank (Fig. 1b). We permute the outputs of conv1 to conv(L−2) of an L-layer network; the last layer, which can never pay, keeps its input. In the maxout and averaging networks, which keep the maximum or mean of maps j and j + N of 2N computed maps, both maps move with π.

Skew (L2). Skewed storage [7] applied to the taps: the host puts channel p of position s in slot $s \cdot N + (p + s) \bmod N$, bank $(p + s) \bmod 4$, which spreads each channel's nine taps 3/2/2/2 over the banks (Fig. 1c). It permutes each window's taps and the consumer's weight columns alike and needs no data and no RTL change; no partial sum, in any order, exceeds $|bias| + K \cdot 127 \cdot 128 < 2^{31}$, so the INT32 result is unchanged. For N = 45, tap $s \cdot 45 + p$ is already in bank $(s + p) \bmod 4$: there L2 = L0.

Skew + relabeling (L3). Under the skew, the channel in position p is in bank $(s + p) \bmod 4$ at 3×3 position s, so a relabeling still decides which channels share a bank at each s; L3 refits it under this rule and then skews (Fig. 1d).

Fitting. Each layer is fitted alone, as relabeled windows are the original's with permuted columns. The recipe draws 2048 windows per fitted layer from Set12 01.png (Cal-1; σ = 25, one noise draw), minimizes their mean of (1) at W = 4, and searches from the identity with 400 proposals, each swapping two channels of different classes (position mod 4), kept only if the objective falls. Under the conventional rule it gave Frozen, the first board layout, and under the skewed rule L3. A second relabeling of the board network, Multi-start, adds Set12 02–03, two noise draws per image (Cal-3) and four random starts (2005 objective evaluations per layer against 401).

Function preserved. In host emulation every folded L1 relabeling reproduced the original's output on Set12 05.png and BSD68 test001.png (Frozen and Multi-start on all 80 images); on every window of every layer of 05.png, on every network, L2 and L3 reproduced the INT32 accumulators (L3 up to its column order) and the folded L3 the output; and the board driver's own skew path gave identical accumulators and PNGs on both images.

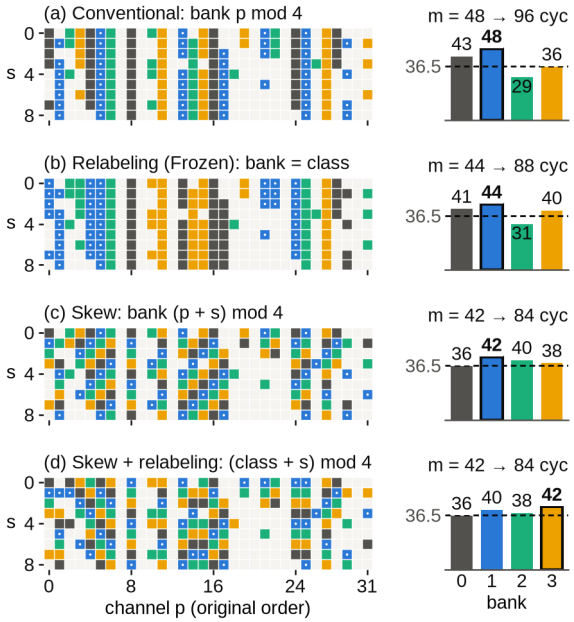


Fig. 1. A conv5 input window of the board network on Set12 05.png (the first compute-bound one at W = 4 with the median m, 48). Filled: nonzero, colored by bank (bank 1 dotted). (a) L0, bank p mod 4; (b) Frozen, an L1 (16 of 32 channels change bank); (c) L2, bank (p + s) mod 4; (d) L3, bank (class + s) mod 4. Bars: nonzeros per bank; the fullest, m, costs 2m cycles; dashed: m = 36.5, where 2m equals the load of 73 cycles. Here L3 ties L2; over all of conv5's windows the mean m is 47.6, 42.1, 40.4 and 39.7.

IV. Experimental Setup

Networks and images. Ten trained integer DnCNN-style networks [9] of six architectures, for Gaussian noise of σ = 25 on 7-bit inputs: A, 7 layers of 32 channels; B, 10 layers of 64 channels behind a 2×2 space-to-depth input; B-32, the same with 32 channels, with ReLU (training seeds s1–s3), maxout over pairs of 64 computed outputs (s1–s3) or their mean (s2); and a 45-channel ReLU variant. All but A had quantization-aware training. The board network, B-32 ReLU s1, reaches 29.54 dB

on Set12 05.png; 37–55% of its hidden layers' window values are nonzero. Test images: Set12 [9] and BSD68 [10], 80 images, one noise draw each; savings are pooled over the 79 other than Set12 01.png, with paired bootstrap 95% intervals over images.

Board. The W = 4 bitstream for the XC7Z020 closes timing at 100 MHz (worst negative slack +0.130 ns), uses 11,151 LUTs (21%), 71 of 140 BRAM36-equivalent tiles and no DSP slice, and ran at the board preset's 50 MHz. One bitstream ran all layouts; per layer the driver checks the output against the integer model and reads the cycle counter CYCLES and the DMA-wait counter IN_STALL. On 05.png the Original ran once as a canary, then each layout three times in alternation. These board cycles are transcribed from run notes, each checked by a script against its cited line, until the raw reports are committed (only an earlier run of the Original is). All other cycle numbers are counts over the windows of the host emulation, which reproduces every network's reference PSNR on all 80 images.

What was fixed when (GitHub push times, UTC). The fill-and-drain rule (6dc0e60, 10-17 13:15) preceded both exports, and Frozen's per-layer count savings on 05.png (9cf9c79, 10-17 18:38) preceded the board runs. PROTOCOL (c2299e2, 10-18 18:32) fixed the recipe, the evaluation, condition (2) per layer and hypotheses H0–H3 before any fit on the other nine networks; PROTOCOL_2 (951c8f1, 10-19 06:07) fixed L2, L3, the bootstrap and H4–H6 before the L3 fit and the study's counts (earlier ones: Sec. VI).

V. Results

A. Board Measurement

On Set12 05.png, Multi-start lowered the board cycles by 7.60% and Frozen by 7.02% (Table I). The repeats gave the same numbers, except 12 more cycles in Frozen's third run, all in conv10, where IN_STALL was also 12; none of the 90 layer runs of the repeats had an output mismatch. In all 63 comparisons (every layer of Original and Multi-start, sums and totals of canary, repeats and the earlier run), CYCLES − IN_STALL equaled the exact prediction. The fill-and-drain rule was read off runs that include the Original's layers, the repeats are deterministic, and conv1 and conv10 see the same windows in every layout; so nine comparisons are out of sample, all on one image: Multi-start's conv2–conv9 and Frozen's conv2–9 sum. This bitstream's IN_STALL misses DMA starvation at window boundaries (fixed in later RTL); the fill-and-drain rule could absorb a stall common to all layouts, so the match excludes only stalls that differ between them. Board runs of L0, Frozen, Multi-start and L2 on Set12 05–08, with every layer's cycles predicted before them (7312a84), are under way, and an addendum (d6521ec) adds L3; no result of them is claimed here.

TABLE I. BOARD CYCLES OF B-32 RELU S1 ON SET12 05.PNG AT W = 4 (REPEAT 1, TRANSCRIBED), EACH EQUAL TO ITS EXACT PREDICTION, AND SAVINGS FROM THEM.

Layout	conv2–9	Total	Saving (%), conv2–9/total
Original	11,421,202	13,289,030	—
Frozen	10,487,986	12,355,814	8.17/7.02
Multi-start	10,410,591	12,278,419	8.85/7.60

B. Ten Networks under Two Protocols

PROTOCOL fixed two checks and two sign predictions, which all held. H0: our code reproduced Frozen in all eight layers, and its export byte for byte. H1: wherever (2) holds, at W = 1 on all networks and at W = 2 on A and B-32 ReLU s1–s3, the relabeled count equaled the original on all 80 images. H2: the pooled W = 4 saving of L1 was positive on all ten networks, 1.09–10.80% (Fig. 2; the sign was known for B, maxout s1 and the board network). H3: the pooled W = 2 saving was positive on all six networks where (2) allows it, 3.36–7.37% on five and 0.022% on the 45-channel network.

PROTOCOL_2 fixed three more, which also held. H4: wherever (2) holds, L2 and L3 counted exactly as L0 on all 80 images, in every layer. H5: at W = 4, L2 saved more than L1 on all nine networks with N mod 4 = 0, by 0.90–2.36 points (known for the board network). H6: L3 saved more than L2 on all nine, by 0.48–1.18 points, and on the board network by 0.73 points (95% interval [0.70, 0.75]): under the skew a relabeling still pays, but far less than alone (7.46%). Not hypotheses: L3 saved more than L2 on each of the 79 images of all nine; at W = 2 the order L3 > L2 > L1 held on the five networks with N mod 4 = 0 where (2) allows a gain, L3 by 0.46–0.85 points over L2; no image counted more cycles under any layout at any W.

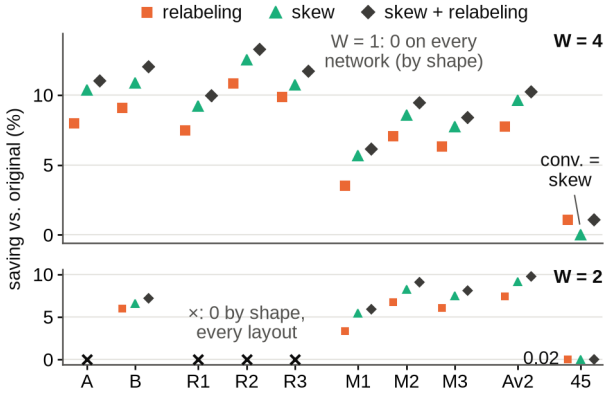


Fig. 2. Pooled saving over the 79 images (count, conv1–convL) at W = 4 (top) and W = 2 (bottom) of L1 (relabeling), L2 (skew) and L3 (skew + relabeling); 95% intervals (at most ±0.39 points) are narrower than the markers. ×: (2) holds in every layer; all layouts count as L0 on all 80 images. On 45 (N mod 4 = 1), L2 = L0 and L3 = L1. A: 7 × 32; B: 10 × 64; R1–R3, M1–M3: B-32 ReLU and maxout, seeds s1–s3 (R1: board network); Av2: averaging, s2; 45: 10 × 45 ReLU.

C. The Board Network on 79 Images

At W = 4, L2 saved more than both board relabelings on each of the 79 images (Fig. 3); L3 reached 82.6% of the perfect-balance saving (Table II). On 05.png the counts give 9.39% (L3) and 8.64% (L2), against 7.02% and 7.60% for Frozen and Multi-start and 11.49% for perfect balance. Pooled, L2 also beat L1 fitted on each test image and a Sense-like sort by each channel's

nonzero count in it; a density deal after SparTen [4] saved least (a comparison of procedures, not of co-activation against density). Multi-start's lead over Frozen comes from Frozen's seed (the worst of five), data and budget, not restarts: greedy search with equal data and budget beat it on four of five seeds.

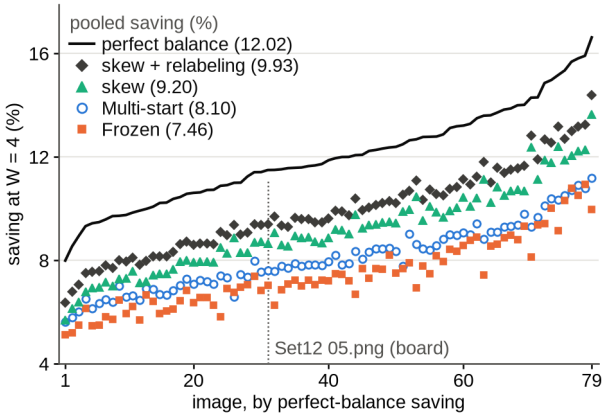


Fig. 3. Per-image W = 4 saving (count) of the board network on the 79 images other than Set12 01.png (calibration of Frozen and L3; Multi-start also used 02–03), sorted by the bound; legend: pooled saving; dotted: the board image.

TABLE II. B-32 RELU S1 AT W = 4 (COUNT, 79 IMAGES): POOLED SAVING AGAINST L0 [95% INTERVAL] AND SHARE (%) OF THE PERFECT-BALANCE SAVING. P.I.: USES THE TEST IMAGE (NO INTERVAL); †: CAL-3 INCLUDES TWO OF THE 79; DENSITY, SENSE-LIKE: BEST OF THREE VARIANTS ON THESE IMAGES.

Layout	Data	Pooled (%)	Share
Density balance [4]	Cal-1	3.69 [3.50, 3.88]	30.7
Sense-like sort [3]	p.i.	4.89	40.7
L1 Frozen	Cal-1	7.46 [7.16, 7.78]	62.1
L1 Multi-start	Cal-3†	8.10 [7.80, 8.39]	67.4
L1 fitted per image	p.i.	8.45	70.3
L2 skew	none	9.20 [8.82, 9.58]	76.6
L3 skew + relabeling	Cal-1	9.93 [9.54, 10.32]	82.6
Perfect balance	—	12.02 [11.59, 12.44]	100

D. Cost

Fitting a network took under 30 s (L1 or L3); L2 needs none. Single-threaded on an x86 server, the host's work for 05.png (input, im2col, requantization) took 31 ms (32 ms on the Multi-start export) against the core's 266 ms at 50 MHz: back to back, Multi-start's 7.60% core saving is 6.3–6.8% end to end, and all of it if the host overlaps the next image; L2's reordering after im2col adds 4.2 ms. This bound omits the transfer of windows and results and the JTAG harness that ran the board (315 s per image, 0.08% of it in the core), and the Zynq's ARM cores would be slower than the server; energy was not measured.

VI. LIMITATIONS

• **Board evidence.** The board measured just two relabelings, on one network and one image; L2 and L3 are counts.

• **Scope and selection.** Only DnCNN-family denoisers at σ = 25, one noise draw per image (the intervals resample images only); BSD68 was part of the networks' development; banks decoupled by FIFOs and other P, T or kernel sizes were not evaluated. Multi-start was chosen for the board after its BSD68 counts had been seen. Before PROTOCOL_2, L2 had been counted on the board network (and on 05.png of four others), and uncommitted counts of L2 on eight images of B and maxout s1 and of L3 on seven of the board network had been seen (PROTOCOL_2 lists them).

VII. CONCLUSION

The layer shapes tell where the tap-to-bank layout of a lockstep zero-skipping core can matter. Where it can, in counts over 79 images, the data-free skew removed more bank imbalance than a fitted relabeling, and a relabeling fitted under the skew removed more still, on all nine networks with N mod 4 = 0. Scripts, layouts, per-image counts and the source of every number are at https://github.com/qewr1234/FPGA (bankperm/).

ACKNOWLEDGMENT

This study was conducted with the support of the Ministry of Science and ICT and the Institute for Information & Communications Technology Planning & Evaluation (IITP) under the 'SW-oriented University Program.'

REFERENCES

[1] J. Albericio, P. Judd, T. Hetherington, T. Aamodt, N. Enright Jerger, and A. Moshovos, "Cnvlutin: Ineffectual-neuron-free deep neural network computing," in *Proc. ACM/IEEE 43rd Annu. Int. Symp. Comput. Archit. (ISCA)*, 2016, pp. 1–13, doi: 10.1109/ISCA.2016.11.

[2] Z.-G. Liu, P. N. Whatmough, Y. Zhu, and M. Mattina, "S2TA: Exploiting structured sparsity for energy-efficient mobile CNN acceleration," in *Proc. IEEE Int. Symp. High-Perform. Comput. Archit. (HPCA)*, 2022, pp. 573–586, doi: 10.1109/HPCA53966.2022.00049.

[3] W. Sun, D. Liu, Z. Zou, W. Sun, S. Chen, and Y. Kang, "Sense: Model-hardware codesign for accelerating sparse CNNs on systolic arrays," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 31, no. 4, pp. 470–483, 2023, doi: 10.1109/TVLSI.2023.3241933.

[4] A. Gondimalla, N. Chesnut, M. Thottethodi, and T. N. Vijaykumar, "SparTen: A sparse tensor accelerator for convolutional neural networks," in *Proc. 52nd Annu. IEEE/ACM Int. Symp. Microarchit. (MICRO)*, 2019, pp. 151–165, doi: 10.1145/3352460.3358291.

[5] J. Pool and C. Yu, "Channel permutations for N:M sparsity," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 34, 2021, pp. 13316–13327.

[6] P. Dang, Y. He, and H. Li, "VARA: A voltage-aware ReRAM-based accelerator for energy-efficient computing," arXiv:2609.00421, 2026.

[7] P. Budnik and D. J. Kuck, "The organization and use of parallel memories," *IEEE Trans. Comput.*, vol. C-20, no. 12, pp. 1566–1569, 1971, doi: 10.1109/T-C.1971.223171.

[8] S. Williams, A. Waterman, and D. Patterson, "Roofline: An insightful visual performance model for multicore architectures," *Commun. ACM*, vol. 52, no. 4, pp. 65–76, 2009, doi: 10.1145/1498765.1498785.

[9] K. Zhang, W. Zuo, Y. Chen, D. Meng, and L. Zhang, "Beyond a Gaussian denoiser: Residual learning of deep CNN for image denoising," *IEEE Trans. Image Process.*, vol. 26, no. 7, pp. 3142–3155, 2017, doi: 10.1109/TIP.2017.2662206.

[10] S. Roth and M. J. Black, "Fields of experts," *Int. J. Comput. Vis.*, vol. 82, no. 2, pp. 205–229, 2009, doi: 10.1007/s11263-008-0197-6.